

Purpose:

- Use the PIC16F883 interrupt system to observe how asynchronous hardware events temporarily change program execution, how multiple interrupt sources share one interrupt vector, and why interrupt service routines must preserve processor context.

- The lab begins with one interrupt source and an observable main program, adds a second source, expands PORTB interrupt-on-change to multiple pins, and then recreates the inherited priority-interrupt behavior with deliberately nested interrupt execution. Mastery reproduces the same visible behavior without nested interrupts or long blocking delays inside an ISR.

- By the end being able to explain what the processor was doing before an interrupt, what caused execution to change, how the source is identified and serviced, what context must be preserved, and how execution safely resumes.

Standards and references:

- RCET 3375 Lab Standard
- RCET PIC-AS Style Guide
- RCET Flowchart Guide
- PIC16F883/882/884/886/887 Datasheet, especially:
 - Section 2.3.2 - Stack
 - Section 3.4.3 - Interrupt-on-change
 - Section 14.3 - Interrupts
 - Section 14.4 and example 14-1 - Context Saving During Interrupts
- PICmicro Mid-Range MCU Family Reference Manual, Section 8 - Interrupts
- previous RCET 3375 lab-book documentation and source for PIC I/O, software delays subroutines, the DLG7137 display, and stack use.
- The PIC16F883 datasheet is the authority for device-specification behavior. Use Section 14.4 and Example 14-1 as the starting point for context save/restore code.

Equipment and materials

- MPLAB X IDE and pic-as toolchain
- PICkit programmer/debugger
- PIC16F883 circuit
- 4MHz crystal oscillator circuit
- momentary pushbutton/switches as required
- LED's and current-limiting resistors
- DLG7137 display used in previous labs
- 4 channel oscilloscope
- logic analyzer, optional
- breadboard, jumpers, and interface components as required
- Lab book

L6

Lab-book documentation:

- For this assignment also include:

- a register map for assigned port pins
- register maps for any GPR's intentionally used as named state/status storage
- datasheets for relevant external components in the lab-book references
- the GitHub URL for your assignment repository in the lab-book references.

- For every interrupt source used, document the enable bit, flag bit, required configuration, what causes the request to be sent^{L6} set, what is required to clear it, and the related SFR settings. When multiple interrupts sources are enabled, each service path must leave its source in a non-requesting state before the common ISR exit.

Part 1 - External Interrupt and Interrupted Main Execution:Goal:

- Configure RBO/INT for a falling-edge interrupt. Each valid interrupt increments PORTC while main toggles a PORTA monitoring pin. Using a signal-sequence oscilloscope capture to observe the interrupt request, interruption of main-line execution, and the resulting count change.

Required behavior -

- Configure RBO/INT for a falling-edge external interrupt.
- Choose any suitable unused PORTA digital output as the main-loop monitoring pin. Document^{L6} the choice.
- Main toggles the selected PORTA pin continuously and as quickly as the program structure reasonably allows.
- Each external interrupt increments PORTC by one
- Display PORTC on LED's as a binary count
- Do not pull RBO/INT from main
- Use the W/STATUS save/restore sequence in PIC16F883 data sheet Section 14.4, example 14-1 as the starting point for processor context handling.
- Mechanical switch bounce is not automatically a failure. If one physical press creates several valid falling edges, observe and document the result.

Before Lab -

- prepare or reference:

- pushbutton, PORTC LED, and PORTA monitoring-point schematic
- electrical/loading-limit analysis
- new SFR maps and relevant bits
- port-pin assignment setup^{L6} map
- External-interrupt setup sequence
- context save/restore plan based on data sheet Example 14-1
- main/interrupt flowchart
- Source code
- predicted oscilloscope behavior.

In the Lab -

- 1) Verify the PORTA monitoring signal.
- 2) Verify the initial PORTC count.
- 3) Press the RBO/INT button and confirm that PORTC increments.
- 4) Observe whether switch bounce creates additional interrupt events.
- 5) Configure a signal-sequence oscilloscope acquisition triggered on the falling edge of RBO/INT
- 6) Capture:
 - CH1: RBO/INT button signal
 - CH2: PORTA Main-loop monitoring pin
 - CH3: RLC, the count LSB
- 7) Identify where main execution is interrupted and where the count changes
- 8) Save and explain the capture in the lab book

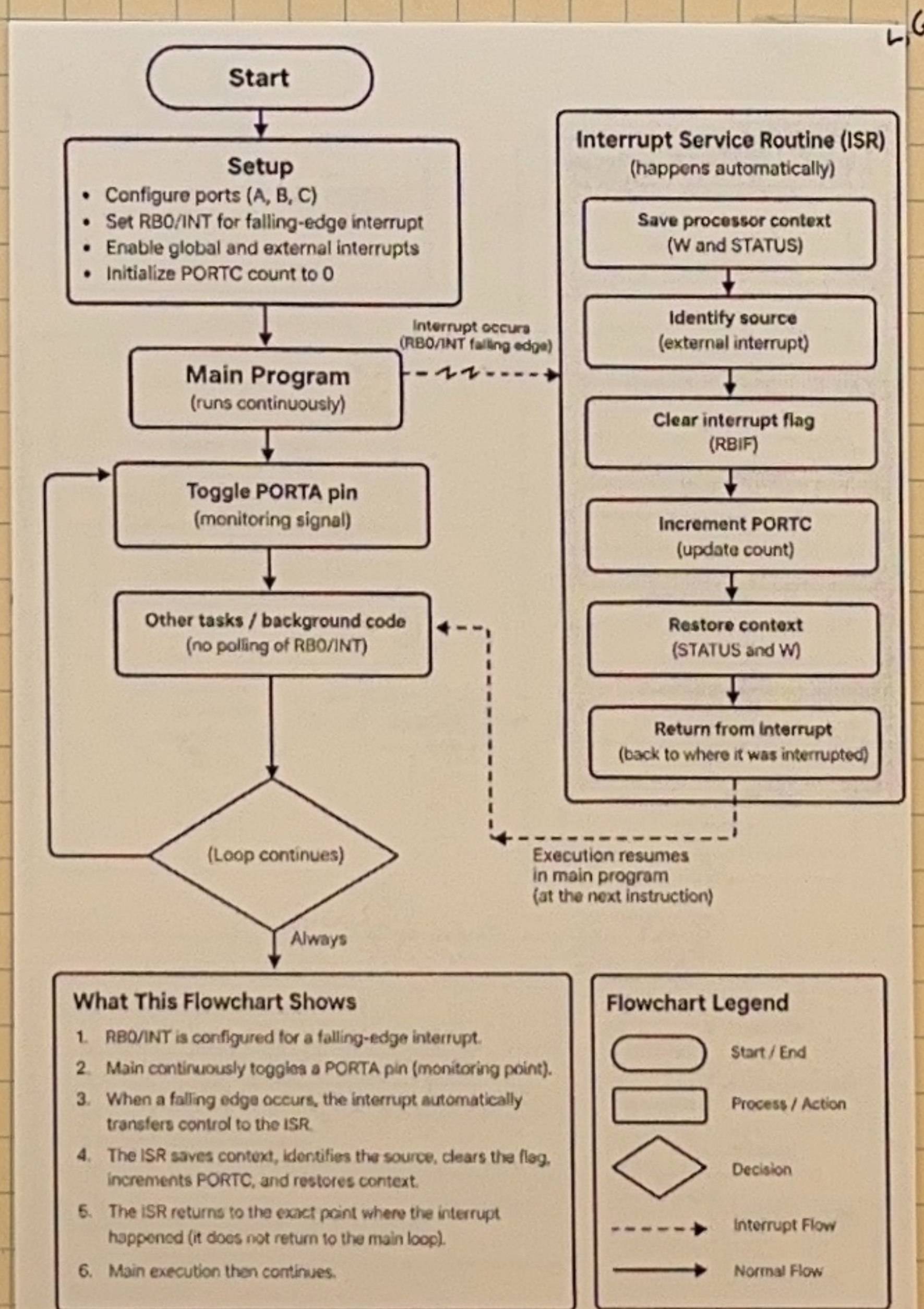


Figure 1: Flow Chart for Part 4 Interrupts

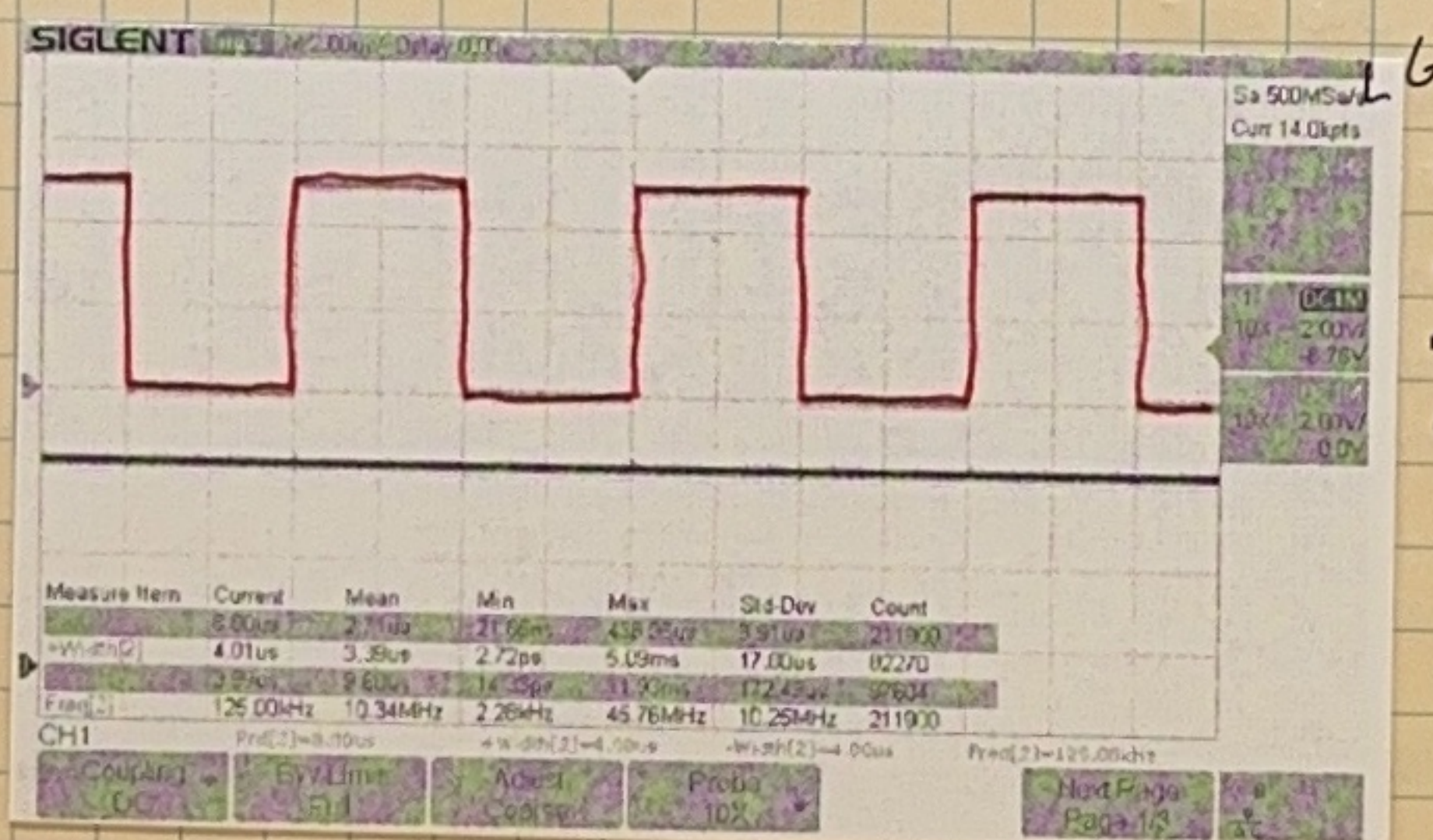
- The new part for this week is the interrupts and ISR's (Interrupt Service Routine). The SFR's show the INTCON's used to enable interrupts and the ISR handler check if RBO has changed and then increments port C's binary count. Within the ISR handler shows how it saves and restores the current state and using the shared registers within all the banks.


```

MOVWF W_TEMP      ;Copy W to TEMP register
SWAPF STATUS,W    ;Swap status to be saved into W
                  ;Swaps are used because they do not affect the status bits
MOVWF STATUS_TEMP ;Save status to bank zero STATUS_TEMP register
;
; (ISR)            ;Insert user code here
;
SWAPF STATUS_TEMP,W ;Swap STATUS_TEMP register into W
                  ; (sets bank to original state)
MOVWF STATUS      ;Move W into STATUS register
SWAPF W_TEMP,F    ;Swap W_TEMP
SWAPF W_TEMP,W    ;Swap W_TEMP into W

```

Figure 5: Saving status and w registers in RAM Doc # DS40001291H-page 214



- PW = 4µs
- PS = 4µs
- Period = 8µs
- Duty cycle % = 50%

Figure 6: Normal operation toggling port A @ 125kHz showing RBO and RAO

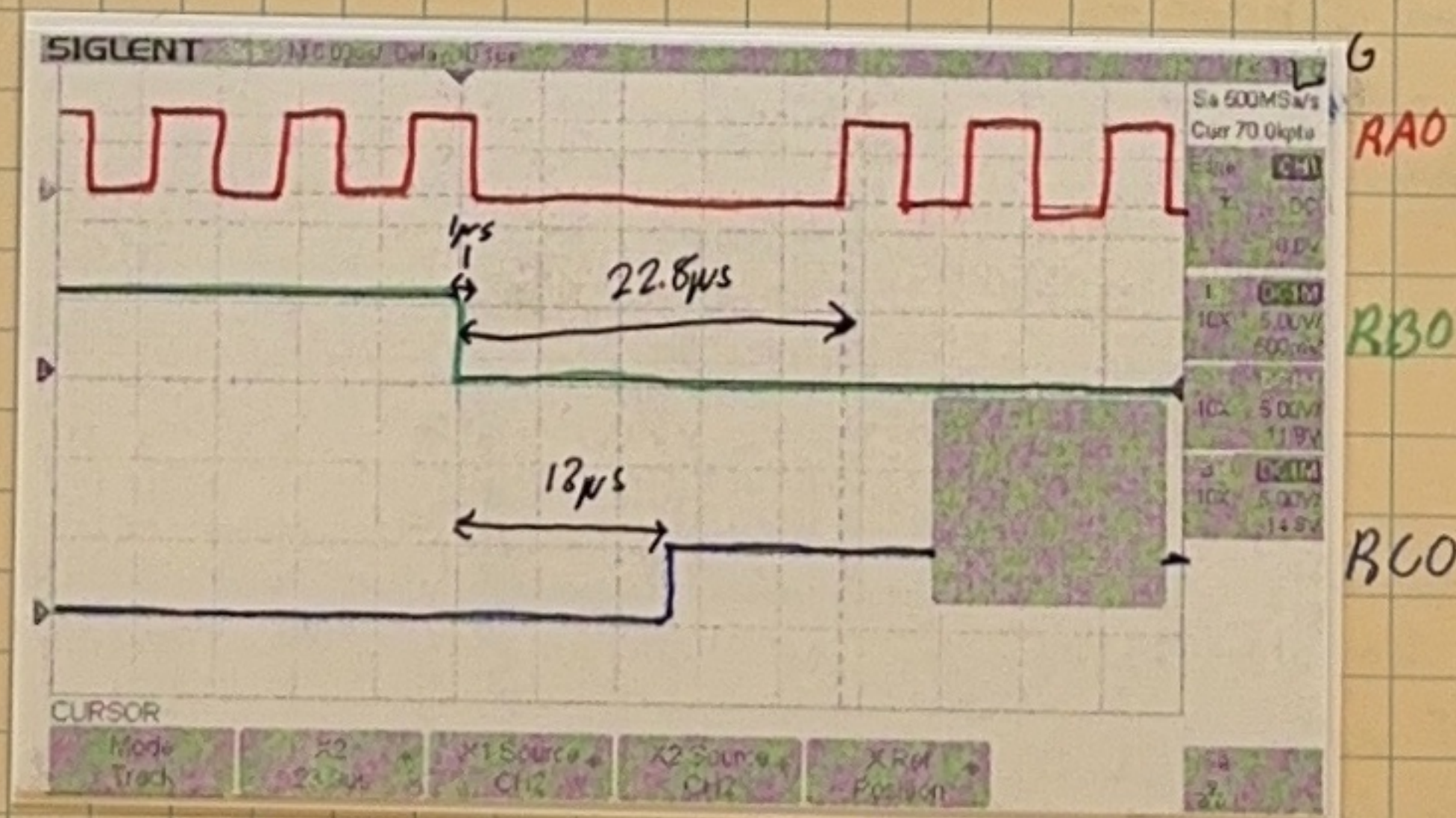


Figure 7: Waveform capture of Interrupt RAO, RBO, & RCO

- This is what the interrupt time looks like and the Tcy of what the interrupt is doing is 2µs. So when it returns the time is where ever it happens, and it starts at that same point.

Lab 5 - Interrupt Service Routine

• Part 2 - Two Interrupt Source and One Interrupt Vector:

• Goal:

- Add RB4 interrupt-on-change while retaining RBO/INT. RB4 IOC increments PORTC; falling edge RBO/INT clears PORTC. The emphasis is source determine, per-source service, and one common ISR exit path.

• Required behavior:

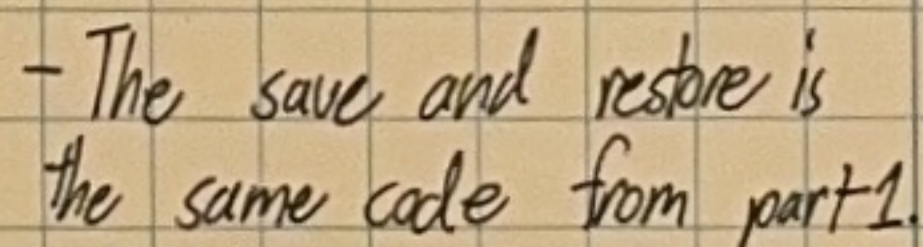
- Retain the PORTA main-loop monitoring signal chosen in Part 1
- Retain PortC LED's as the binary count
- Configure RB4 as the IOC input
- An RB4 change event increments PORTC
- A falling-edge RBO/INT event clears PORTC to zero
- Main polls neither sources
- Both sources enter through the same interrupt vector.
- Save context once, determine the source, service it, and restore context once at the common exit before RETFIE.
- Each service path must resolve and clear its own interrupt condition before leaving for the common exit.
- Correctly resolve the PORTB mismatch/change condition before clearing the IOC flag.

• Before Lab:

- update schematic with RBO/INT and RB4 IOC inputs
- IOC SRT SFR documentation
- update flow port pin assignment map
- flowchart showing on vector, source determination, two service paths, and one exit
- context save/common exit structure
- source code
- predicted behavior when events occur close together.

• In the Lab:

- 1) Verify that RB4 IOC increments PORTC
- 2) Generate several RB4 changes and confirm that the count accumulates
- 3) Trigger RBO/INT and confirm that PORTC clears
- 4) Test events occurring close together
- 5) Keep these 4 signals connected to the oscilloscope
 - CH1: RBO/INT
 - CH2: PORTA main loop monitoring pin
 - CH3: RCO
 - CH4: RB4 IOC input
- 6) Capture an RB4 IOC event in a signal sequence mode and explain what's happening.
- 7) Set the PORTC count on RAO so RCO High, then capture an RBO/INT event in signal single sequence and explain.
- 8) If useful, capture a close event sequence with both sources occurring near one another and compare it with predicted behavior.

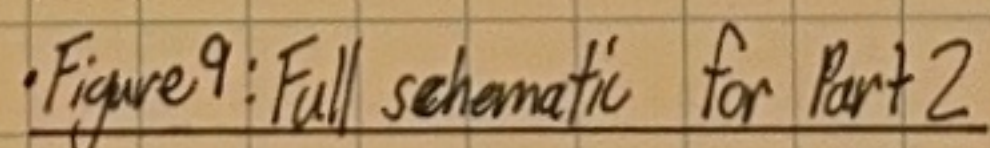


- Trigger: change on RB4
- Flag: RBIF (INTCON<0>)
- Enable: RBIE (INTCON<3>)
- Clear: read PORTB, then clear RBIF
- Action: increment PORTC

Flowchart Legend

	Start / End
	Process / Action
	Decision
	Interrupt Flow
	Normal Flow

Figure 8: Flow chart for Part 2 interrupt for clear and count interrupt



- The schematic's only change is a second button.

Part 3- Determine Which PORTB Pin ChangedGoal:

- Use interrupt on change on all eight PORTB pins and determine in software which pin changed.
Display the detected pin number in binary on three PORTA LED's.

Required behavior:

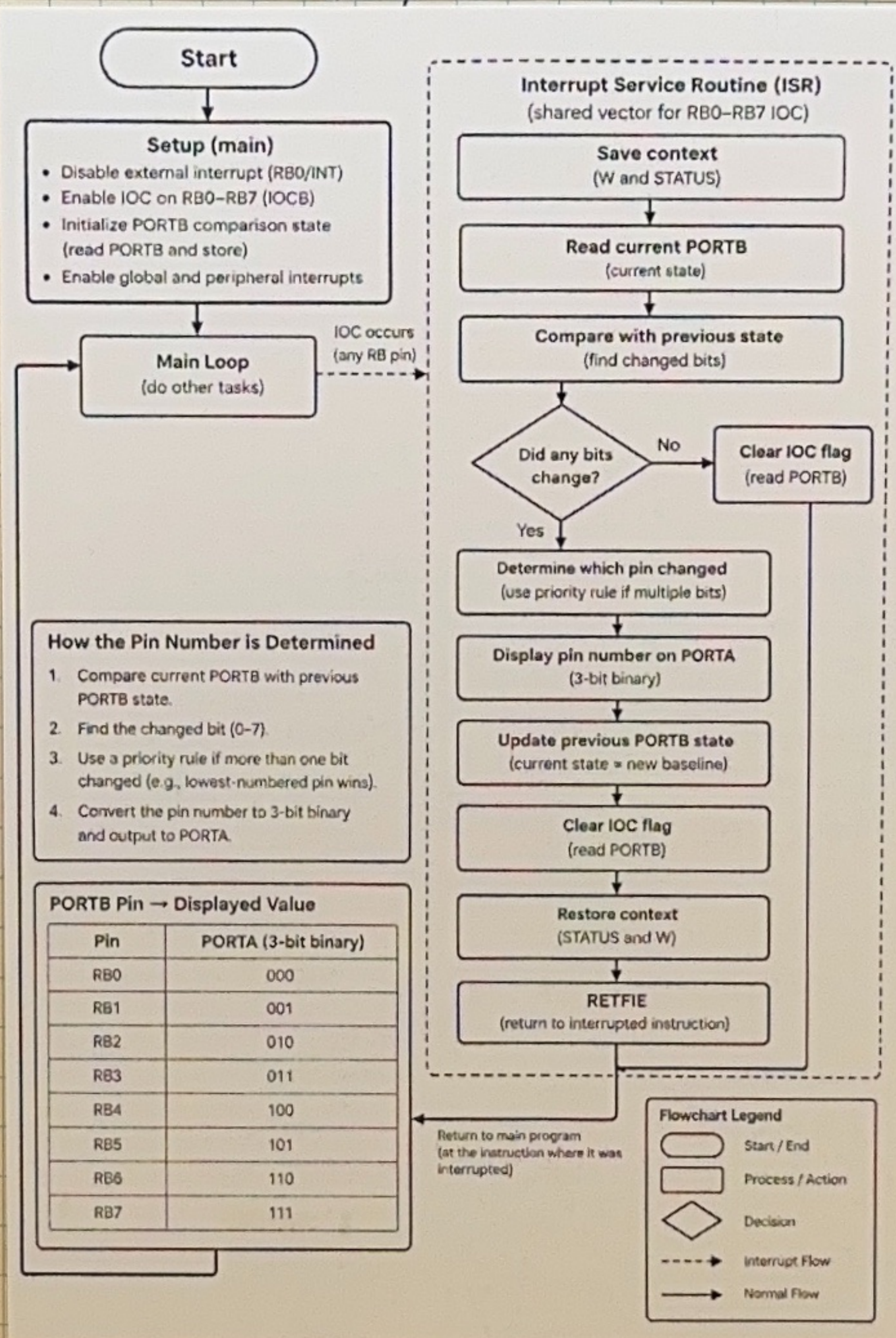
- Disable the dedicated external interrupt for this part
- Enable IOC for RB0 through RB7 using IOCB
- Establish the initial PORTB comparison state before enabling IOC
- Treat the IOC flag as notification that a PORTB change occurred, not as identification of a particular pin.
- Determine which PORTB bit change from the current and previous port states or an equivalent method.
- Display the pin number on three PORTA outputs:
 - RB0 = 000
 - RB1 = 001
 - ⋮
 - RB7 = 111
- Define a deterministic rule for the case where more than one changed bit is present when the ISR evaluates the event.
- Correctly update the PORTB state used for comparison.
- Correctly service and clear IOC before leaving the handler.

Before Lab:

- PORTB input and PORTA LED schematic
- IOC configuration documentation
- PIN assignment map
- source determination algorithm
- flow chart
- deterministic multi-change rule
- source code
- register-style map for named state/status storage used by the design.
- the effect of the PICkit/ICSP connection on RB6 and RB7 while those pins are used as IOC.

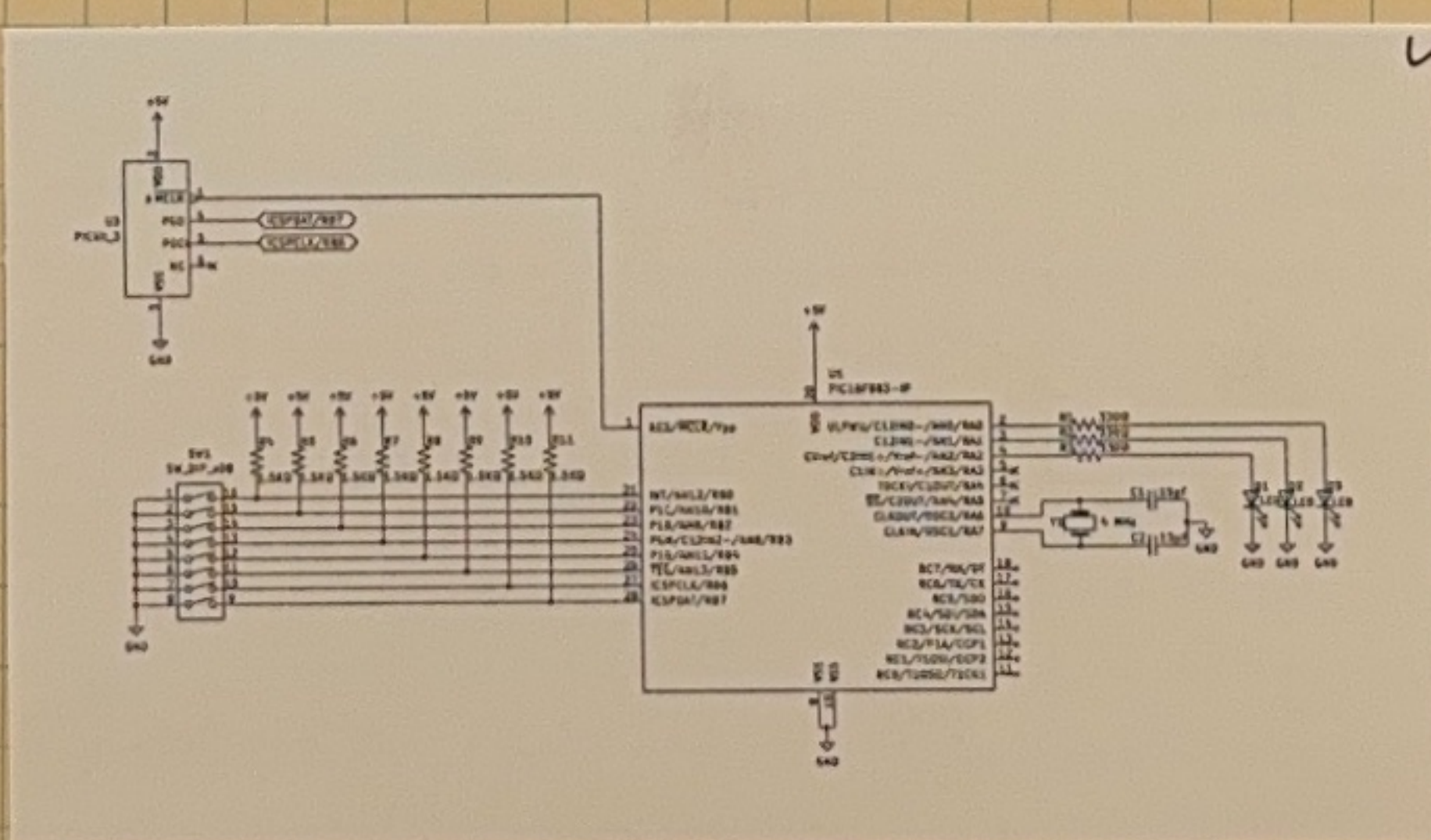
In the Lab:

- 1) Verify RB0 through RB7 individually
- 2) Confirm the correct three-bit pin number for every input.
- 3) Test repeated changes on the same pin
- 4) Test different pin in sequence
- 5) Create at least one condition where more than one change may be present & verify the documented rule.
- 6) confirm that the ISR doesn't become stuck servicing an uncleared IOC condition.
- 7) Record any effect of bounce or closely spaced input changes on source determination.
- 8) Verify RB6 & RB7 with the PICkit/ICSP connection in the configuration intend to use during normal operation.



• Main really just sits there and waits for a change on PORTB and then displays that RB inputs value in binary count on PORTA. Then clears the interrupt on change flag and remembers the PORTB value so it doesn't through another flag.

Figure 14: Flow chart for Part 3 PORTB Inputs & PORTA LED's outputs



• The 8 Dip switch is used to set the input values. Then RA0-RA2 is used to display the binary count. RA0 = LSB & RA2 = MSB.

Figure 15: Full schematic for Part 3 PORTB inputs & PORTA LED's

Lab 5 - Interrupt Service Routine

9/23/2026

LR6

Setup:

```

; Bank 1
BANKSEL TRISA
CLRF TRISA ;Set RA's as output for monitoring binary count
CLRF TRISC ;PORTC left as before but not used
MOVLW 0xFF
MOVWF TRISB ;set all of PORTB inputs } make Port B inputs
MOVLW 0xFF
MOVWF IOCB ;enable IOC on RB0-RB7 } set all Port B to interrupt on change

; Bank 3
BANKSEL ANSEL
CLRF ANSEL ;set PortA as digital I/O } set PORTA & B as digital I/O's
CLRF ANSELH ;set PortB as digital I/O

; Bank 0
BANKSEL PORTA
CLRF PORTA ;monitor pin starts low } set PORTA Low.
MOVF PORTB, W ;initial comparison state } start with 0 for comparison.
MOVWF RB_OLD ;before enabling the interrupt
BCF INTCON, 0 ;clear change on PortB interrupt flag
BSF INTCON, 3 ;enable PortB change interrupt
BSF INTCON, 7 ;global enable } set interrupts

```

Main:

GOTO Main

Figure 16: Setup and "main code" for Part 2

```

; read current PORTB and find what changed
MOVF PORTB, W ;W will be the current PORTB
MOVWF RB_NEW ;save it, you'll need it twice
XORWF RB_OLD, W ;W = RB_NEW XOR RB_OLD changed bits
MOVWF RB_CHANGED

; update the comparison state for next time
MOVF RB_NEW, W
MOVWF RB_OLD

; determine which bit changed -- lowest pin wins
BTFSC RB_CHANGED, 0 ;if RB0 is low skip GOTO next test
GOTO Pin0 ;if RB0 is high move the binary value to portA
BTFSC RB_CHANGED, 1 ;if RB1 is low skip GOTO next test
GOTO Pin1 ;if RB1 is high move the binary value to portA

```

This part checks the old and moves that new value that was changed and store it on the RB_CHANGED.

update the RB_OLD with the new value.

This goes to the value to display.

Figure 17: Select why the interrupt happened and display its value.

```

Pin6:
MOVLW 0x06 ;move 110 to PortA
GOTO SetPin

Pin7:
MOVLW 0x07 ;move 111 to PortA

SetPin:
MOVWF PORTA ;display the value in binary } display value

IOC_Done:
BCF INTCON, 0 ;clear RBIF PORTB was already re-read } clear flag

ISRExit:
; restore
SWAPF STATUS_TEMP, W ;un-swap STATUS into W
MOVWF STATUS ;restores flags AND bank bits (RPO/RP1)
SWAPF W_TEMP, F ;swap W_TEMP nibbles in place
SWAPF W_TEMP, W ;swap back into W, STATUS untouched
RETFIE } restore

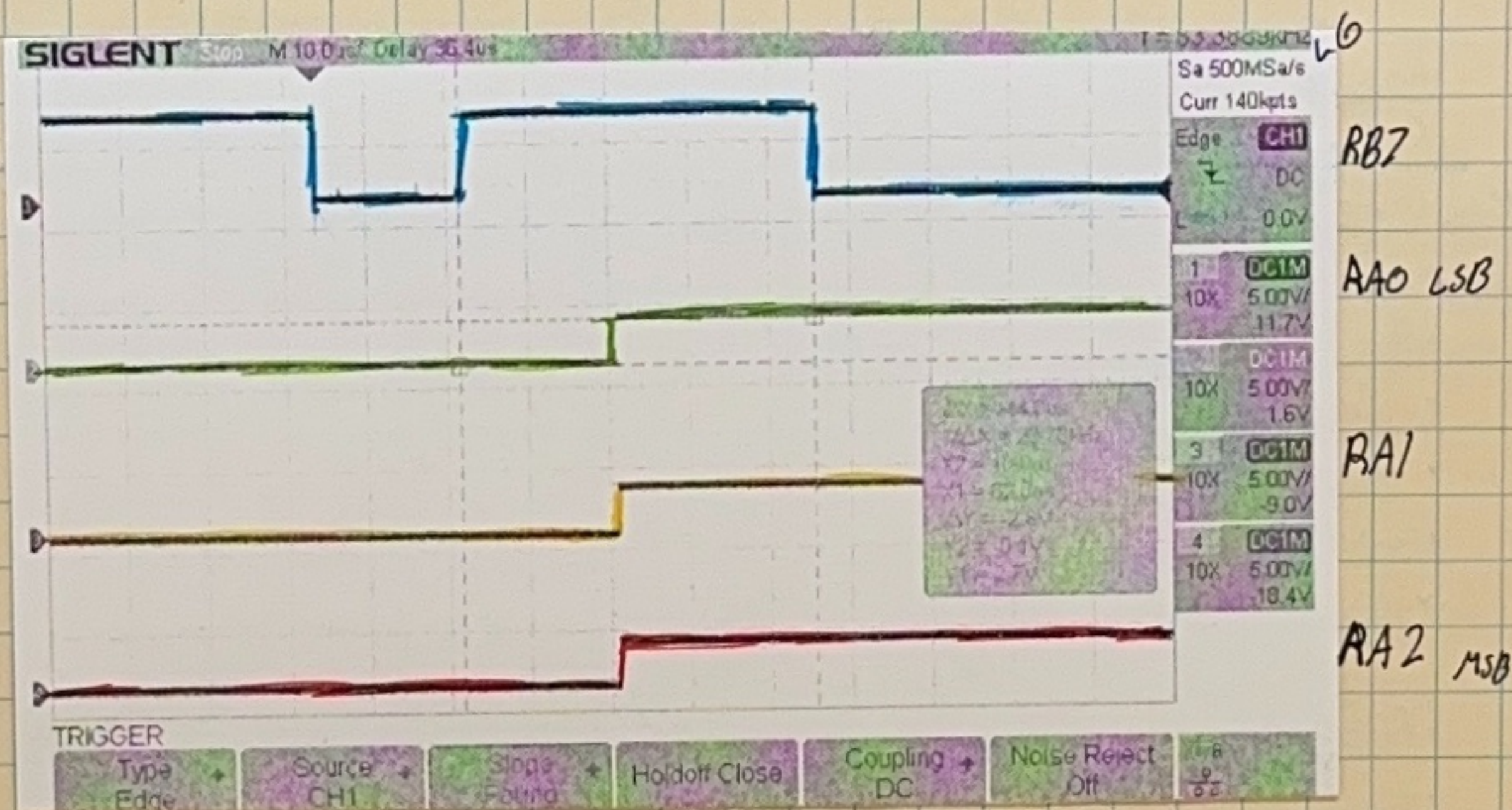
```

Figure 18: Display and restore the flag and restore

9/23/2026

LR6

• This section all calculations for parts see pg 66 part 1 figure 2. The purpose was to treat the IOC flag as a notification and take that information and figure out ^{L6} how to display the correct value from an input and move it to an output.



• Figure 11: Capture of setting RB7's change and PORTA RA1, 2, & 3

- This is when it is set low so a change and displays the interrupts effect. So even if it was already set High and another was switched it only cares about its next change value. The High part is a bounce

TJR
9-25-26

• Part 4 - Priority Interrupts, Nested Execution, and Stack Depth:

• Goal:

- Create 1/6/7 priority-interrupt behavior using the DL67137 display on PORTC from the earlier I/O labs. This part intentionally allows the higher-priority 7 behavior to interrupt the lower priority 6 behavior so you can investigate nested interrupt execution, subroutines calls, context preservation, and worst case hardware stack depth.

• Required visible behavior:

- The display on PORTC must precisely display 1, 6, & 7 at any time to reflect the current state of the system.
- Main displays 1 continuously
- Falling-edge RBO/INT initiates the 6 behavior.
- IOC on RB1 initiates the 7 behavior
- 6 displays for 2 seconds.
- 7 displays for 2 seconds has priority
- 7 may interrupt 6 at any time while 6 is active
- While 7 is active, no lower-priority behavior may interrupt it
- When 7 finishes, execution returns to the state that was active when 7 began.
- If 7 interrupted 6/6 resumes and completes its remaining time before returning to 1.
- If 7 interrupted the normal 1 state, the display returns to 1.
- 7 may interrupt the same 6 behavior multiple times. Each time, 6 resumes its remaining time after 7 finishes.

• Delay and subroutine requirement:

- Implement the long delays with callable subroutines using CALL/RETURN. Use nested delay/subroutine call as appropriate. Normal subroutines calls are intentionally part of the stack depth problem in this part.

• Required stack analysis:

- Before demonstration, determine the worst-case hardware stack depth of the final design. Account for every return address that can be active in the worst case including.

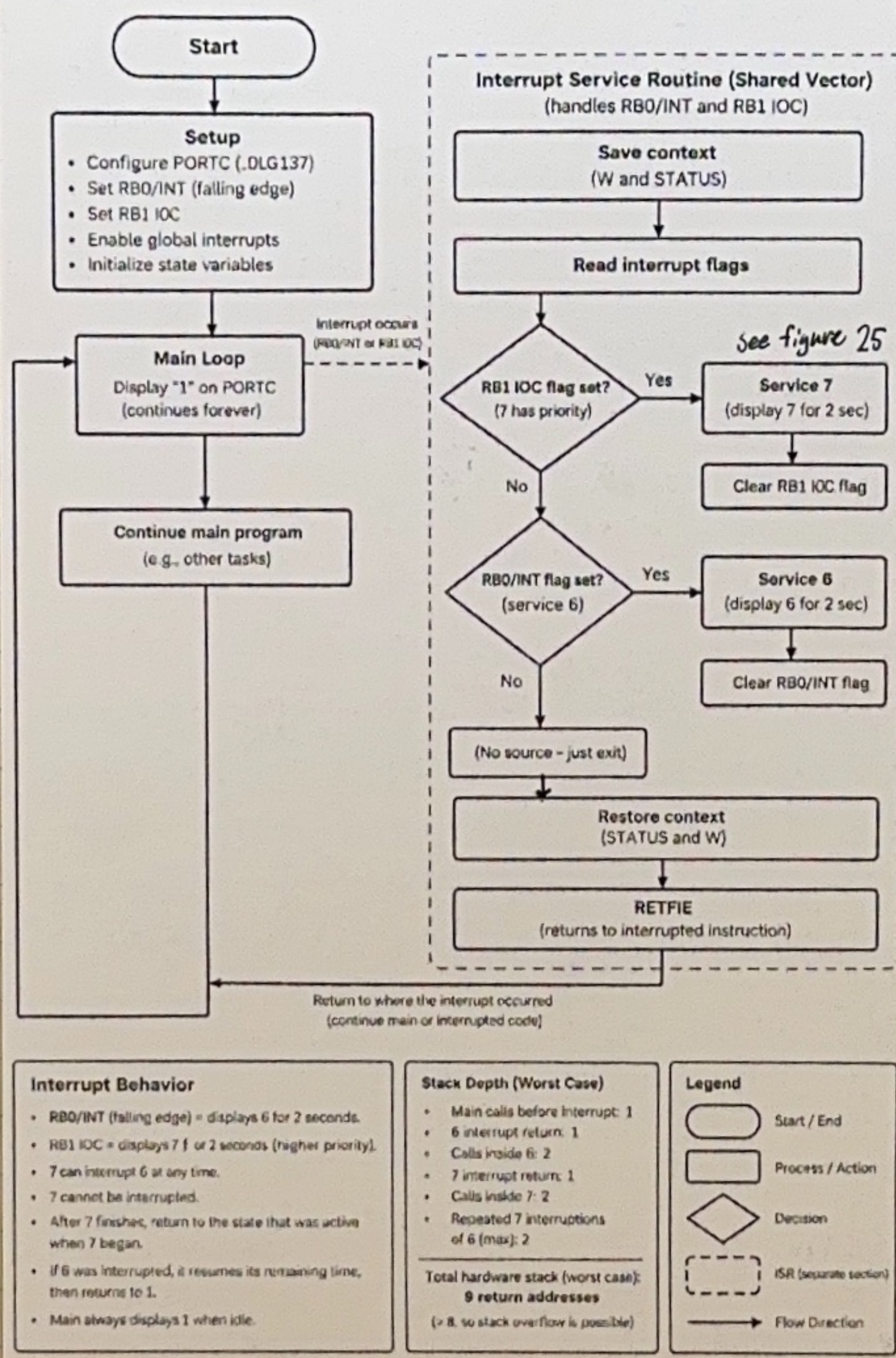
- call depth before the first interrupt
- the 6 interrupt return address
- calls made while executing the 6 behavior
- the nested 7 interrupt return address
- calls made while executing the 7 behavior
- repeated 7 interruptions of 6 as allowed by the design.

• Before Lab:

- earlier DLG7137/PORTC schematic, ^{LG}end encoding, and component data sheet
- RBO external interrupt and RB1 IOC wiring
- interrupt/IOC SFR documentation
- pin assignment map
- nested interrupt flowchart
- nested context save strategy
- delay subroutines structure
- worst case hardware stack analysis
- source code

• In the Lab:

- 1) Verify that main displays 1.
- 2) Trigger RBO/Int and verify the approximately two second 6 behavior followed by a return to 1.
- 3) Trigger RB1 IOC while 1 is active. Verify 1 → 7 → 1.
- 4) Trigger 7 while 6 is active. Verify 6 → 7 → 6 → 1, with 6 completing the time that remained when it was interrupted.
- 5) Trigger 7 repeatedly at different points during one active 6 behavior. Each 7 must complete without interruption, then return to the interrupt 6 until 6 eventually completes and returns to 1.
- 6) Attempt to trigger 6 while 7 is active and verify that 6 doesn't interrupt 7.
- 7) Compare observed execution with the stack depth analysis.
- 8) The instructor will intentionally exercise the design near its worst case nesting condition and may attempt to expose stack or context problems.
- 9) Record and explain any failure observed.



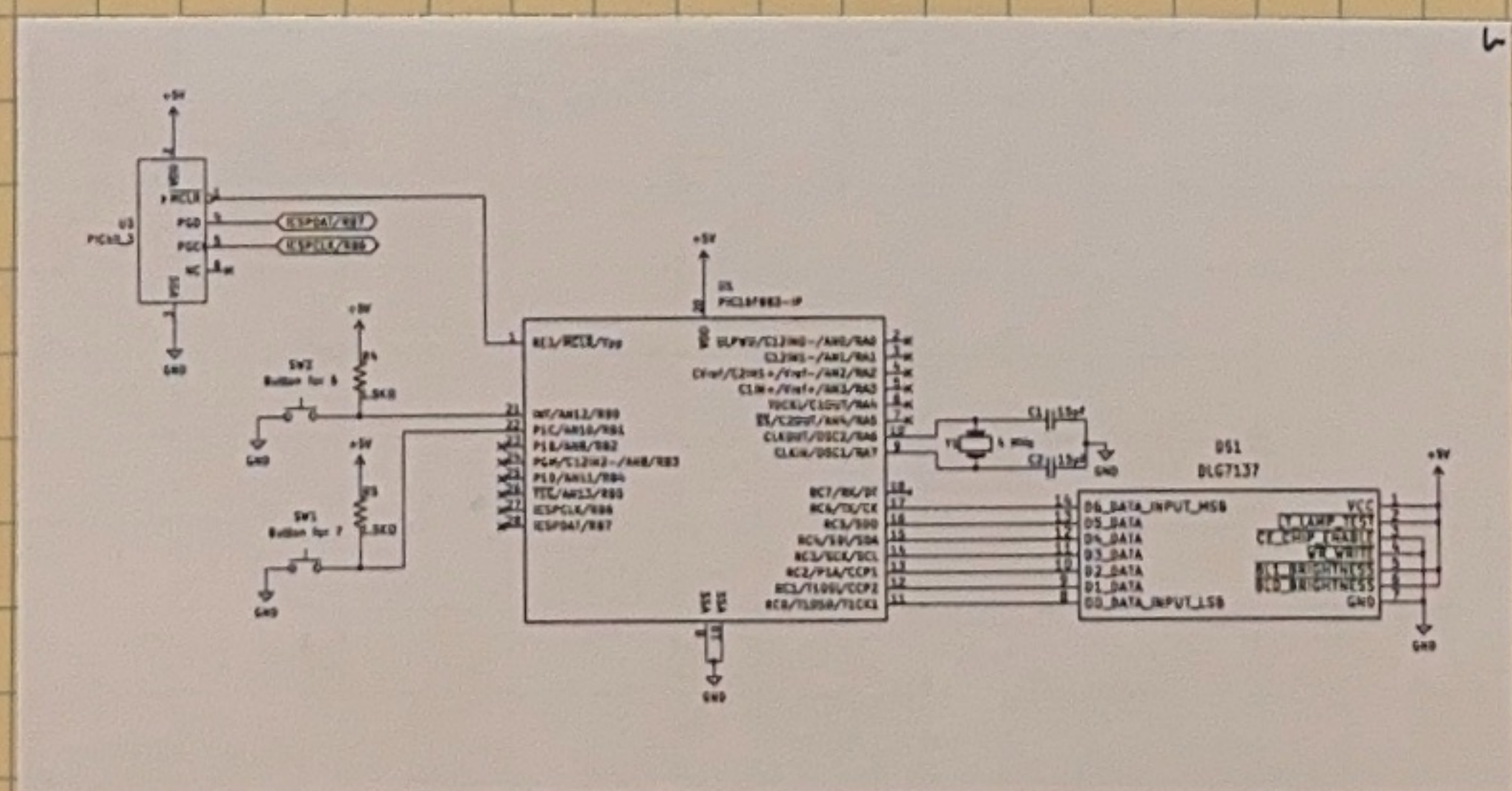
• Main's only job is to just display 1 unless an interrupt happens then it will display the priority bit.

• RB1 the interrupt on change bit sets the flag which is what the code looks for when checking what runs. Highest priority of display 7

• RB0 is the external interrupt flag bit. which is for display and delay of 6.

• Both 6 & 7 have 2 sec delays if 7 interrupts 6. 7 will display and the 6 will finish its delay.

• Figure 20: Flowchart for Part 4 on Priority Interrupts, Nested Execution, & Stack Depth



• Figure 21: Full schematic for part 4 priority interrupts

• RB0 is connected to button 2 and when it is pressed it will display 6 on the DLG137 unless 7 has been pressed. Button 1 is for the highest priority of displaying 7.

```

#include <xc.inc>

W_TEMP_A      EQU 0x70 ; this is for the LG storage when the interrupt is not
STATUS_TEMP_A EQU 0x71 ; already in "6"
W_TEMP_B      EQU 0x72 ; used when 7 interrupts 6 while is running.
STATUS_TEMP_B EQU 0x73 ; holds the status of when 6 is interrupted by 7
FLAGS         EQU 0x74 ; remembers the old display value
OLD_DISPLAY   EQU 0x75

maincount6    EQU 0x20 ; set the main delay for display 6
outcount6     EQU 0x21 ; set the outer delay for display 6 } used for the display
incount6      EQU 0x22 ; set the inner delay for display 6 } 6 delay
maincount7    EQU 0x23 ; set the main delay for display 7
outcount7     EQU 0x24 ; set the outer delay for display 7 } used for the display
incount7      EQU 0x25 ; set the inner delay for display 7 } 7 delay

```

Figure 22: The extra registers used to save memory and counts in the interrupts.

```

Setup:
; Bank 1
BANKSEL TRISA
CLRF TRISC ; PORTC set as outputs for display
BSF TRISB,0 ; set RB0 of PORTB to input
BSF TRISB,1 ; set RB1 of PORTB to input
BSF IOCB,1 ; enable IOC on RB1
BCF OPTION_REG,6 ; RB0/INT falling edge

; Bank 3
BANKSEL ANSEL
CLRF ANSEL ; set PortC as digital I/O
CLRF ANSELH ; set PortB as digital I/O

; Bank 0
BANKSEL PORTB
MOVF PORTB,W ; initial comparison state
CLRF FLAGS ; start with nothing in the flags register
BCF INTCON,0 ; clear change on PortB interrupt flag } • set up interrupts.
BCF INTCON,1 ; clear external interrupt flag
BSF INTCON,3 ; enable PortB change interrupt
BSF INTCON,4 ; enable external interrupt bit
BSF INTCON,7 ; enable global interrupt bit

Main:
MOVLW 0x31 ; Select the character set for display 1 } keeps 1 always displayed
MOVWF PORTC ; Send W to PortC } unless interrupted.
GOTO Main

```

Figure 23: SFR / Setup code for the display and inputs for the interrupts

```

ISRHandler:
BTFSC FLAGS,0 ; if already in 6 save the context in B this is for a nested }
GOTO SaveOuterB ; and if not inside 6 save context in A,

SaveOuterA:
MOVWF W_TEMP_A ; save W (no flags affected)
SWAPF STATUS,W ; STATUS into W with nibbles swapped, no flags change
MOVWF STATUS_TEMP_A ; save swapped STATUS
GOTO CheckSource

SaveOuterB:
MOVWF W_TEMP_B ; save W (no flags affected)
SWAPF STATUS,W ; STATUS into W with nibbles swapped, no flags change
MOVWF STATUS_TEMP_B ; save swapped STATUS

```

Figure 24: The save memory for both interrupts and current status

```

CheckSource:
;      RBI IOC = display 7
BTFSS  INTCON,0      ;check the interrupt on change bit flag
GOTO   CheckINT      ;if not set, check the other source

;      display 7
MOVWF  PORTB,W       ;dummy read - this is what actually clears the mismatch
BCF    INTCON,0       ;clear the IOC flag right now
MOVWF  PORTC,W       ;grab whatever's currently displayed
MOVWF  OLD_DISPLAY   ;this will be 1 or 6 depending on what 7 interrupted
MOVLW  0x37
MOVWF  PORTC         ;put 7 on the display
BCF    INTCON,7       ;keep interrupts off during the delay
CALL   LongDelay7     ;2 seconds, uninterruptible
MOVWF  OLD_DISPLAY,W  ;restore whatever was showing before 7 started
MOVWF  PORTC
GOTO   ISRExit

```

check to see if it was 6 not 7 was selected.

Figure 25: Code Ran if RB1 or PORTB interrupt on change bit is selected.

```

LongDelay7:
MOVLW  0x22 1tcy = 24
MOVWF  maincount7 1tcy
MainDelayLoop7:
MOVLW  0x32 1tcy = 50
MOVWF  outcount7 1tcy
OutDelayLoop7:
CALL   ShortDelay7 2tcy
DECFSZ outcount7,1 1(2)tcy
GOTO   OutDelayLoop7 2tcy
DECFSZ maincount7,1 1(2)tcy
GOTO   MainDelayLoop7 (2)tcy
RETURN 2tcy

ShortDelay7:
MOVLW  0xC8 1tcy =
MOVWF  incount7 1tcy
InDelayLoop7:
DECFSZ incount7,1 1(2)tcy
GOTO   InDelayLoop7 2tcy
RETURN 2tcy

```

$K = 34$
 $M = 50$
 $N = 200$

$3N + 4$
 $(N(2+1) + 3) + 2$
 $(37 + (3N + 5) + 5) \times 5 + 6 = 2007 \text{ sec.}$

Figure 26: Delay nested loop for RB1 selected bit for a 2 second delay

```

ISRExit:
BTFSC  FLAGS,0      ; still inside 6's window? then THIS exit is 7's
GOTO   RestoreInner ; exit, and it must restore slot B, not slot A

RestoreOuter:
SWAPF  STATUS_TEMP_A,W ;restore
MOVWF  STATUS
SWAPF  W_TEMP_A,F
SWAPF  W_TEMP_A,W
RETFIE ; pops the return address, GIE turns back on

RestoreInner:
SWAPF  STATUS_TEMP_B,W
MOVWF  STATUS
SWAPF  W_TEMP_B,F
SWAPF  W_TEMP_B,W
RETFIE

```

6 was selected and must restore

7 if 6 was interrupted. and must restore to continue.

Figure 27: Restore saved data before leaving the ISR Handler

- The worst case stack depth has an 8 level hardware stack that only stores return address. 6 levels is at most the deepest this code will get.

1) Main is running = 0 levels used here

2) RBO interrupt occurs = 1 level used before returning to main

3) Inside the 6 ISR called LongDelay6 = 2 levels deep ^{LB} deep

4) LongDelay6 calls ShortDelay6 = 3 levels deep

5) While inside ShortDelay6 a 7 interrupt occurs = 4 levels deep before returning to shortDelay6.

6) Inside the 7 ISR called LongDelay7 = 5 levels deep

7) LongDelay7 calls ShortDelay7 = 6 levels deep

- The Context save works for the nested execution uses temp paired registers first the a slot which is used for a normal interrupt but the b slot is used when 7 interrupts while 6 is running. The Flag register keeps track of whether a 6 is currently active.

- Trouble shooting came to a timing issue for display 7 even though both 6 and 7 delays were the same 7 was longer almost double so by halving the long delay it ^{LB} brought it back.

LB

9/25/2026

LRG

Lab 5-Interrupt Service Routine

81

Conclusion: This weeks lab was understanding ISR's and how they work and what

they do⁶ during the operation. Part 1 was setting RBO/INT to be configured for a falling edge, so that PORTA could toggle. Learning and using context saving and restoring for Docu# DS4000129/H-219. Part 2 was covering RB4 IOC (Interrupt on Change) was added as well on the same vector so the ISR handler had to check if it will increment or clear by checking which interrupt occurred. RB4's IOC design was to increment portC while RBO will clear PORTC all using one ISR handler. Part 3 was using a dig switch on portB to check which pin was selected and where it was an interrupt on change whether high or low any change caused portA to display which button or value changed. Part 4 is the 6/7 lab using the DLG7137 from PORTC using a priority and delay. 7 has the highest priority and can overwrite 6. This made it so there was ~~no~~ need for 2 ⁶ways to save and restore. All this is shared in one vector this worked because the button for 7 always clears the global interrupt so nothing can overwrite it. Also requires 2 different delays for each. This used calls and made us go deep into our stack (my code went 6 deep). Most problems came from mixing up which flag to clear and understanding which variable that need to stay separate.

TSR
9-25-26

L6

9/25/2026

LRG